# Python: exercises on loops with strings and lists

This notebook was generated in **pseudocode** mode.

---

## Exercise 1: Shortest Words

Exercise Description

Shortest Words

Write a function `shortest_words` that:

- takes a list of words
- returns a list containing the shortest words in the list received as argument. The list will containt more than one word when there are multiple words with the same length.

Examples:

- `shortest_words([])` returns `[]`
- `shortest_words(['sheldon', 'cooper'])` returns `['cooper']`
- `shortest_words(['sheldon', 'cooper', 'howard'])` returns `['cooper', 'howard']`

**NOTE**: do not use any buit-in function from Python to solve the exercise

Your solution

```
# Define function shortest_words that takes parameter lista_stringhe
    # Set lunghezza_minima to positive infinity (very large number)
    # For each parola in lista_stringhe:
        # Set lunghezza_parola to the length of parola
        # If the value is less than the threshold:
            # Set lunghezza_minima to lunghezza_parola
    # Initialize parole_corte as an empty list
    # For each parola in lista_stringhe:
        # If the values are equal:
            # Add parola to the parole_corte list
    # Return the result: parole_corte
```

Test your solution

```
assert shortest_words([]) == []
assert sorted(shortest_words(['sheldon', 'cooper'])) == ['cooper']
assert sorted(shortest_words(['sheldon', 'cooper', 'howard'])) == ['cooper', 'howard'
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 2: find how many equal numbers

Exercise Description

find how many equal numbers

Define a function `count_equals` that:

- takes as arguments four numbers
- returns:
    - the maximum number of equal numbers between the four

Example:

- `count_equals(1,2,3,4)` should return `0`
- `count_equals(1,2,5,4)` should return `0`
- `count_equals(1,2,2,2)` should return `3` because there are three `2` in the sequence
- `count_equals(1,1,1,2)` should return `3` because there are three `1` in the sequence

Your solution

```
def count_equals(a, b, c, d):
    # step 1: create a list of the four numbers
    # step 2: initialize a variable to store the maximum count of equal numbers
    # step 3: iterate over the list of numbers
        # step 4: for each number:
            # count how many times it appears in the list
            # update the maximum count if the current count is greater
    # step 5: return the maximum count
```

Test your solution

```
assert count_equals(1,2,3,4) == 0
assert count_equals(1,5,3,4) == 0
assert count_equals(1,5,5,4) == 2
assert count_equals(1,5,1,4) == 2
assert count_equals(1,5,5,5) == 3
```

Test your solution

```
# Test case 2 (assert)
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 3: Fibonacci's sequence

### Exercise Description

Fibonacci's sequence

Define a function `fibonacci` that:

- takes as arguments:
    - an integer number `n`
- returns:
    - a list containing the first `n` numbers of the Fibonacci's sequence

**NOTE**: The Fibonacci sequence is a series of numbers where each number is the sum of the two previous ones, starting with 0 and 1. To calculate it, you begin with 0 and 1, then add

these to get the next number. Continue this process to generate the sequence. It goes 0, 1, 1, 2, 3, 5, 8, and so on.

Your solution

```
def fibonacci(n):
    # step 1: initialize the list with the first two numbers of the sequence
    # step 2: if n is 1, return the list
    # step 3: iterate from 2 to n-1
        # step 4: for each iteration, calculate the next number as the sum of the las
        # step 5: append the new number to the list
    # step 6: return the list
```

Test your solution

```
assert fibonacci(1) == [0]
assert fibonacci(3) == [0,1,1]
assert fibonacci(7) == [0, 1, 1, 2, 3, 5, 8]
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 4: zero-sum triplets

Exercise Description

zero-sum triplets

Define a function `zero_sum_triplets` that:

- takes as arguments:
    - a list of integers `numbers`
- returns:
    - the number of triplets whose sum is zero

Example:

- `zero_sum_triplets([1,-1,0,7,12])` should return `1` because the sum of 1,-1,0 is `0`
- `zero_sum_triplets([1,9,0,7,12])` should return `0` because there are no triplets that sum up to zero
- `zero_sum_triplets([1,-9,8,6,-14])` should return `2` because the sum of 1,-1,0 is `0` and the sum of 8,6,-14 is `0`

Your solution

```
def zero_sum_triplets(numbers):
    # step 1: initialize a variable to store the count of zero-sum triplets
    # step 2: iterate over the list of numbers
        # step 3: for each number, check if it can be part of a zero-sum triplet
            # step 4: if it can, increment the count
    # step 5: return the count
```

Test your solution

```
assert zero_sum_triplets([1,-1,0,7,12]) == 1
assert zero_sum_triplets([1,9,0,7,12]) == 0
assert zero_sum_triplets([1,-9,8,6,-14]) == 2
assert zero_sum_triplets([0, 0, 0]) == 1
assert zero_sum_triplets([5, -5, 0, 10, -10]) == 2
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

# Exercise 5: Collatz

Exercise Description

Collatz

Define a function `collatz` that:

- takes as argument an integer number `n`
- returns:
    - a list containing all the numbers generated by the Collatz conjecture (stopping when reaching `1`)

**NOTE**: The Collatz Conjecture is a mathematical problem that starts with any positive integer. The process involves two steps: if the number is even, divide it by 2; if it's odd, multiply it by 3 and add 1. Repeat this process with the resulting number. The conjecture suggests that, no matter what number you start with, you'll eventually reach the number 1.

Your solution

```
def collatz(n):
    # step 1: initialize an empty list to store the sequence
    # step 2: while n is not 1
    #     - step 2.1: append n to the sequence
    #     - step 2.2: if n is even, update n to n // 2
```

```
    #     - step 2.3: if n is odd, update n to 3 * n + 1
    # step 3: return the sequence
```

Test your solution

```
assert collatz(12) == [6,3,10,5,16,8,4,2,1]
assert collatz(1) == []
assert collatz(2) == [1]
assert collatz(4) == [2,1]
assert collatz(19) == [58, 29, 88, 44, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16,
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 6: Find missing number in a sequence

Exercise Description

Find missing number in a sequence

Define a function `find_missing` that:

- Takes a list of `n-1` integers, which represents a sequence of numbers from 1 to `n`, but one number is missing.
- Returns the missing number.

**NOTE**: Suppose that there is always only one number missing

Example:

- `find_missing([1, 2, 4, 5])` should return `3`.
- `find_missing([2, 3, 4, 6, 1])` should return `5`.

Your solution

```
def find_missing(nums):
    # step 1: calculate the expected length of the complete list, which includes one
    # step 2: calculate the total sum of the first 'n' natural numbers using the form
    # step 3: calculate the actual sum of the numbers present in the input list 'nums
    # step 4: the missing number is the difference between the total sum and the actu
    # step 5: return the missing number
```

Test your solution

```
assert find_missing([1, 2, 4, 5]) == 3
assert find_missing([2, 3, 4, 6, 1]) == 5
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 7: Prefixes of a String

Exercise Description

1. Define the string `s` equal to `The Big Bang Theory`
2. Create the empty list `p`
3. Using a loop, add all prefixes of `s` to `p`
4. Print `p`

Your solution

```
# assign value to variable
# assign value to variable
# iterate through sequence
    # add element to list
# print the result
```

Test your solution

```
assert len(p) == 19
```

Test your solution

```
assert p[0] == "T"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 8: Check Palindrome

Exercise Description

Check Palindrome

Define a function `is_palindrome` that:

- Takes a string as input.
- Returns `True` if the string is a palindrome, and `False` otherwise.

**NOTE**: A palindrome is a word, phrase, number, or other sequence of characters that reads the same forward and backward (**ignoring spaces, punctuation, and capitalization**).

Example:

- `is_palindrome("racecar")` should return `True`.
- `is_palindrome("hello")` should return `False`.
- `is_palindrome("A man, a plan, a canal, Panama")` should return `True`.

Your solution

```
def is_palindrome(s):
    # Step 1: Initialize an empty string 's2' to hold the filtered and normalized cha
    # Step 2: Iterate over each character in the input string 's'
        # Step 3: Check if the character is an alphabetic character
            # Step 4: Convert the character to lowercase and add it to 's2'
    # Step 5: Check if the filtered string is a palindrome by comparing characters
```

```
        # Step 6: If characters at symmetric positions do not match, it's not a palin
    # Step 7: If all symmetric characters match, return True indicating it's a palind
```

Test your solution

```
assert is_palindrome("racecar")
assert not is_palindrome("hello")
assert is_palindrome("A man, a plan, a canal, Panama")
```

Test your solution

```
# Test case 2 (assert)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 9: Check Postfixes of a String

Exercise Description

1. Define the string `s` equal to `The Big Bang Theory`
2. Define the list `p` equal to `["y", "ry", "ery", ""]`
3. Remove from `p` any string that is not a postfix of `s`
4. Print `p`

Your solution

```
# assign value to variable
# assign value to variable
# assign value to variable
# iterate through sequence
    # check condition
        # add element to list
# iterate through sequence
    # remove element from list
# print the result
```

Test your solution

```
assert p == ["y", "ry", ""]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 10: Prime Numbers

Exercise Description

Define a function `is_prime` that:

- takes as arguments a list `L` of positive integers
- returns:
    - if the list is not empty: a new list where the i-th element is a boolean asserting whether the i-th element from `L` is a prime number
    - otherwise: `[]`

Your solution

```
def is_prime_number(n):
    # check condition
        # return the result
    # iterate through sequence
        # check condition
            # return the result
    # return the result

def is_prime(L):
```

```
    # assign value to variable
    # iterate through sequence
        # add element to list
    # return the result
```

Test your solution

```
assert is_prime([]) == []
```

Test your solution

```
assert is_prime([3, 4, 9, 11]) == [True, False, False, True]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

---

# Exercise 11: Word Frequency

Exercise Description

Define a function `count_freq` that:

- takes as arguments:
    - a string `s`
    - a list `L` of words
- returns:
    - if the list is not empty: a new list where the i-th element is the number of occurences in `s` of the i-th word from `L`
    - otherwise: `[]`

Your solution

```
def count_freq(s, L):
    # assign value to variable
    # iterate through sequence
        # add element to list
    # return the result
```

Test your solution

```
assert count_freq("test", []) == []
```

Test your solution

```
assert count_freq("Aejeje", ["e", "je", "aje"]) == [3, 2, 0]
```

Debug your solution